Python & Algorithms for Coding Interviews

Section 3B — Core Algorithmic Patterns (3.11 – 3.20)

3.11 Backtracking

The Problem Situation

You need to generate all possible combinations, subsets, permutations, or find all paths satisfying a constraint. Backtracking explores every possibility but prunes branches that cannot lead to a valid answer, making it far more efficient than brute force enumeration.

Key Insight

Insight	Build the solution incrementally. At each step, make a choice and recurse. If the current path is invalid or complete, undo the choice (backtrack) and try the next option. The key operation is: choose → explore → unchoose.

Analogy

Mental Model	Solving a Sudoku puzzle. You write a number in a cell, then try to fill the rest. If you get stuck (a cell has no valid number), you erase your last choice and try the next number. You explore all possibilities but stop exploring any path the moment it becomes invalid.

Basic Code Template

```
# Universal backtracking template
def backtrack(state, choices, result):
    if is_complete(state):      # base case
        result.append(list(state))
        return
    for choice in choices:
        if is valid(state, choice):  # prune invalid paths
            state.append(choice)     # choose
            backtrack(state, choices, result)  # explore
```

```
            state.pop()              # unchoose (backtrack)

# Subsets — include or exclude each element
def subsets(nums):
    result = []
    def bt(start, path):
        result.append(list(path))
        for i in range(start, len(nums)):
            path.append(nums[i])
            bt(i + 1, path)
            path.pop()
    bt(0, [])
    return result
```

✅**This Template Solves**	LeetCode 78 — Subsets • LeetCode 90 — Subsets II • LeetCode 77 — Combinations

Alternative Template

```
# Permutations
def permutations(nums):
    result = []
    def bt(path, remaining):
        if not remaining:
            result.append(list(path))
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            bt(path, remaining[:i] + remaining[i+1:])
            path.pop()
    bt([], nums)
    return result

# Combination Sum — reuse elements, sum to target
def combination sum(candidates, target):
    result = []
    def bt(start, path, remaining):
        if remaining == 0:
            result.append(list(path)); return
        if remaining < 0: return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            bt(i, path, remaining - candidates[i])  # i not i+1 (reuse
allowed)
            path.pop()
    bt(0, [], target)
    return result
```

✅**This Template Solves**	LeetCode 46 — Permutations • LeetCode 39 — Combination Sum • LeetCode 131 — Palindrome Partitioning

Example Walkthrough

```
# Example: subsets([1,2,3])
# bt(0,[]): append [] → try 1
#   bt(1,[1]): append [1] → try 2
#     bt(2,[1,2]): append [1,2] → try 3
#       bt(3,[1,2,3]): append [1,2,3]
#     pop 3 → pop 2 → try 3
#     bt(3,[1,3]): append [1,3]
#   pop → try 2,3 similarly
# Result: [[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]
```

Time & Space Complexity

Case	Time	Space
Subsets	O(2^n)	O(n)
Permutations	O(n!)	O(n)
Combinations	O(C(n,k))	O(k)

Pattern Recognition Clues

- Keywords: all combinations, all subsets, all permutations, generate all, find all paths
- You must enumerate all valid configurations — backtracking is the default tool
- Decision tree — at each node you make a binary choice (include/exclude) or k-ary choice
- Constraint satisfaction — Sudoku, N-Queens, word search on grid

Interview Traps

□ **Trap**	Appending the path object directly (result.append(path)) — appends a reference, not a copy. Always append list(path).
□ **Trap**	Not incrementing start index — causes duplicates in combination problems
□ **Trap**	Missing the pruning condition — without pruning, backtracking is just brute force

Edge Cases

- Empty input — return [[]] for subsets, [] for permutations
- Duplicates in input — sort first and skip same-value siblings (LeetCode 40, 90)
- Target = 0 in combination sum — empty combination is valid

Important Practice Problems

- LeetCode 78 — Subsets
- LeetCode 46 — Permutations
- LeetCode 39 — Combination Sum
- LeetCode 79 — Word Search
- LeetCode 131 — Palindrome Partitioning

Real World Python Usage

Real World	Backtracking powers constraint solvers in AI planning systems. SAT solvers (used in chip design verification) use DPLL — a backtracking algorithm. Python's re module uses backtracking for regex matching. Puzzle solvers (Sudoku, crosswords) in apps use backtracking.

3.12 Recursion and Divide & Conquer

The Problem Situation

A problem can be broken into smaller subproblems of the same type. Solving each subproblem independently and combining results gives the final answer. Merge Sort, Quick Sort, and most tree problems use this pattern. Recursion is the mechanism; Divide & Conquer is the strategy.

Key Insight

Insight	Divide the problem into smaller subproblems. Conquer each subproblem recursively. Combine the results. The base case is the smallest problem you can solve directly. The recursive case reduces the problem size each call, guaranteeing termination.

Analogy

Mental Model	Breaking a large team project into subtasks. Each subtask is assigned to a smaller team (recursive call). When each subtask is done, the results are merged into the final deliverable. The smallest subtask (a single person's work) is the base case.

Basic Code Template

```
# Recursive function structure
def solve(problem):
    # Base case — smallest solvable problem
```

```
    if is_base_case(problem):
        return base_solution
    # Divide
    left_part, right_part = divide(problem)
    # Conquer
    left_result  = solve(left_part)
    right_result = solve(right_part)
    # Combine
    return combine(left result, right result)

# Merge Sort — classic divide and conquer
def merge sort(nums):
    if len(nums) <= 1:
        return nums
    mid   = len(nums) // 2
    left  = merge_sort(nums[:mid])
    right = merge_sort(nums[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j  = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]: result.append(left[i]);  i += 1
        else:                   result.append(right[j]); j += 1
    return result + left[i:] + right[j:]
```

✅**This Template Solves**	LeetCode 912 — Sort an Array • LeetCode 23 — Merge K Sorted Lists • LeetCode 240 — Search a 2D Matrix II

Alternative Template

```
# Understanding the call stack during recursion
# fibonacci(4):
#   fibonacci(3)
#     fibonacci(2)
#       fibonacci(1) → 1
#       fibonacci(0) → 0
#     return 1
#     fibonacci(1) → 1
#   return 2
#   fibonacci(2)
#     fibonacci(1) → 1
#     fibonacci(0) → 0
#   return 1
# return 3

# Without memoization: O(2^n) — same subproblems computed repeatedly
# With @lru_cache:     O(n)   — each subproblem computed once
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

✅**This Template Solves**	LeetCode 169 — Majority Element • LeetCode 53 — Maximum Subarray (D&C approach) • LeetCode 215 — Kth Largest Element

Example Walkthrough

```
# Example: Maximum Subarray via Divide & Conquer
# nums = [-2,1,-3,4,-1,2,1,-5,4]
# Divide at mid=4:
#   left  = [-2,1,-3,4,-1]  → max crossing includes index 4 (-1)
#   right = [2,1,-5,4]      → max crossing includes index 5 (2)
# Cross-max: scan left from mid → right from mid+1
# left_sum: -1,3,0,-2,-4  → max=3
# right_sum: 2,3,-2,2     → max=3
# cross = 3+3 = 6
# Answer: max(left_result, right_result, cross) = 6
```

Time & Space Complexity

Case	Time	Space
Merge Sort	O(n log n)	O(n)
Binary Search (D&C)	O(log n)	O(1)
Tree recursion (balanced)	O(n)	O(log n)

Pattern Recognition Clues

- Keywords: tree problems, sort, find Kth, majority, largest subarray — often D&C
- Problem structure naturally splits into independent halves
- When memoization would help — recursive tree with repeated subproblems → add @lru_cache
- Recursion on a tree is D&C — solve left subtree, solve right subtree, combine

Interview Traps

⬜ **Trap**	Missing base case — infinite recursion and stack overflow
⬜ **Trap**	Not reducing problem size in recursive call — call must get strictly smaller each time
⬜ **Trap**	Forgetting @lru_cache on recursive solutions with overlapping subproblems — O(2^n) vs O(n)

Edge Cases

- n = 0 or 1 — base cases must handle all minimal inputs
- Very deep recursion on large n — switch to iterative or increase recursion limit
- Overlapping subproblems — add memoization to avoid recomputation

Important Practice Problems

- LeetCode 912 — Sort an Array
- LeetCode 23 — Merge K Sorted Lists
- LeetCode 169 — Majority Element
- LeetCode 215 — Kth Largest Element in an Array
- LeetCode 53 — Maximum Subarray

Real World Python Usage

Real World	Python's built-in sort (Timsort) uses a hybrid of merge sort and insertion sort. NumPy's FFT uses divide and conquer on signal arrays. MapReduce (Hadoop, Spark) is an engineering-scale divide and conquer paradigm. Distributed systems split large datasets using recursive partitioning.

3.13 Dynamic Programming (DP)

The Problem Situation

A problem has two properties: overlapping subproblems (the same smaller problem is solved multiple times) and optimal substructure (the optimal solution to the whole problem contains optimal solutions to subproblems). DP solves each subproblem exactly once and stores the result — converting exponential time to polynomial time.

Key Insight

Insight	Identify what changes between subproblems — that is your state. Define dp[state] = answer for that state. Write a recurrence relation: dp[state] = f(dp[smaller_state]). Either use top-down memoization (@lru_cache) or build bottom-up from the smallest state to the largest.

Analogy

□ Mental Model	Climbing stairs. You need to reach step n. You know you can reach step n from step n-1 or n-2. So you just need to know: how many ways to reach n-1 and n-2? You store those answers so you never compute them twice. Each step's answer builds on the previous steps' answers.

Basic Code Template

```
# Top-Down DP — Memoization with @lru_cache
from functools import lru_cache

# Fibonacci — classic DP
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

# Coin Change — min coins to make amount
@lru_cache(maxsize=None)
def coin_change_memo(coins, amount):
    if amount == 0: return 0
    if amount  < 0: return float('inf')
    return 1 + min(coin_change_memo(coins, amount - c) for c in coins)
```

✓This Template Solves	LeetCode 70 — Climbing Stairs • LeetCode 322 — Coin Change • LeetCode 1143 — LCS

Alternative Template

```
# Bottom-Up DP — Tabulation
# Coin Change iterative
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0   # base case: 0 coins to make amount 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], 1 + dp[a - coin])
    return dp[amount] if dp[amount] != float('inf') else -1

# Longest Common Subsequence (LCS) — 2D DP
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

```
# 0/1 Knapsack
def knapsack(weights, values, capacity):
    n  = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            dp[i][w] = dp[i-1][w]  # don't take item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], values[i-1] + dp[i-1][w-weights[i-
1]])
    return dp[n][capacity]
```

✅This Template Solves	LeetCode 322 — Coin Change • LeetCode 1143 — LCS • LeetCode 300 — LIS • LeetCode 416 — Partition Equal Subset Sum

Example Walkthrough

```
# Recurrence relations — the heart of DP
# Climbing Stairs:   dp[i] = dp[i-1] + dp[i-2]
# Coin Change:       dp[a] = min(dp[a-coin]+1) for coin in coins
# LCS:               dp[i][j] = dp[i-1][j-1]+1   if match
#                             = max(dp[i-1][j], dp[i][j-1]) if no match
# 0/1 Knapsack:      dp[i][w] = max(dp[i-1][w],
#                                   val[i]+dp[i-1][w-wt[i]])
# LIS:               dp[i] = max(dp[j]+1) for j<i if nums[j]<nums[i]

# Common DP subpatterns:
# Linear DP      → dp[i] depends on dp[i-1] or dp[i-k]
# 2D DP          → dp[i][j] depends on dp[i-1][j], dp[i][j-1]
# Interval DP    → dp[i][j] = best way to solve subarray [i..j]
# DP on strings  → LCS, edit distance, palindrome
# Knapsack       → include/exclude each item, capacity constraint
```

Time & Space Complexity

Case	Time	Space
Top-down memo	O(states)	O(states)
Bottom-up tab	O(states)	O(states)
Space optimised (rolling)	O(states)	O(1 row)

Pattern Recognition Clues

- Keywords: minimum/maximum, count ways, is it possible, optimal, partition
- Overlapping subproblems — same inputs computed multiple times in naive recursion

- Optimal substructure — optimal answer uses optimal answers to subproblems
- Choices at each step — include/exclude, take/skip, move direction

Interview Traps

□ **Trap**	Incorrect base cases — dp[0] and dp[1] must be set carefully before the loop
□ **Trap**	Wrong transition direction — ensure dp[i] only depends on already-computed states
□ **Trap**	2D DP off-by-one — dp is size (m+1)×(n+1) but strings are indexed 0 to m-1

Edge Cases

- Amount = 0 or empty input → should return 0 or empty
- No valid solution exists → return -1 or float('inf')
- Duplicate elements in knapsack inputs
- Very large state space — consider space optimisation with rolling array

Important Practice Problems

- LeetCode 70 — Climbing Stairs
- LeetCode 322 — Coin Change
- LeetCode 300 — Longest Increasing Subsequence
- LeetCode 1143— Longest Common Subsequence
- LeetCode 416 — Partition Equal Subset Sum

Real World Python Usage

□ **Real World**	DP powers autocorrect and spell check (edit distance). Sequence alignment in bioinformatics (DNA matching) uses LCS-style DP. Python's difflib module uses DP for text diff. Video codec compression (H.264) uses DP for motion estimation. Google's PageRank can be formulated as a DP on a graph.

3.14 Greedy Algorithms

The Problem Situation

At each step you make the locally optimal choice — the choice that looks best right now — without reconsidering previous choices. Greedy works when the local optimal choice at every step leads to the globally optimal solution. It does not work when local decisions have downstream side effects that need to be reconsidered.

Key Insight

Insight	Make the best possible decision at every step and never look back. Greedy works when the problem has the greedy choice property: choosing the locally best option at each step leads to the globally best outcome. When this property holds, greedy is far simpler and faster than DP.

Analogy

Mental Model	Giving change with the fewest coins (using large coins first) — works with standard coin denominations. At each step, take the largest coin that fits. This works because standard coins (1,5,10,25 cents) have a special mathematical property. With arbitrary coin values, this greedy fails — you would need DP.

Basic Code Template

```
# Activity Selection — greedy by earliest end time
def activity_selection(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by end time
    count, last_end = 1, intervals[0][1]
    for start, end in intervals[1:]:
        if start >= last_end:            # non-overlapping
            count   += 1
            last_end = end
    return count

# Jump Game — can you reach the end?
def can_jump(nums):
    max_reach = 0
    for i, jump in enumerate(nums):
        if i > max_reach: return False   # stuck
        max_reach = max(max_reach, i + jump)
    return True
```

This Template Solves	LeetCode 55 — Jump Game • LeetCode 435 — Non-overlapping Intervals • LeetCode 45 — Jump Game II

Alternative Template

```
# Interval scheduling — merge or count overlapping intervals
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
```

```
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:              # overlapping
            merged[-1][1] = max(merged[-1][1], end)   # extend
        else:
            merged.append([start, end])
    return merged
```

✅**This Template Solves**	LeetCode 56 — Merge Intervals • LeetCode 57 — Insert Interval • LeetCode 253 — Meeting Rooms II

Example Walkthrough

```
# Example: Non-overlapping Intervals
# intervals = [[1,2],[2,3],[3,4],[1,3]]
# Sort by end: [[1,2],[2,3],[1,3],[3,4]]
# Keep [1,2]: last_end=2
# [2,3]: start(2) >= last_end(2) → keep, last_end=3
# [1,3]: start(1) < last end(3)  → REMOVE (overlap)
# [3,4]: start(3) >= last_end(3) → keep
# Removed 1 interval → answer = 1
```

Time & Space Complexity

Case	Time	Space
Interval sorting	O(n log n)	O(1) or O(n)
Single pass greedy	O(n)	O(1)

Pattern Recognition Clues

- Keywords: minimum number of steps, maximum non-overlapping, earliest deadline, least cost
- Sorting by a greedy criterion (end time, value/weight ratio) often reveals the greedy order
- If DP feels like overkill and the problem has a natural ordering — try greedy first
- Interval problems almost always have a greedy solution

Interview Traps

⚠ **Trap**	Applying greedy when it does not have the greedy choice property — leads to wrong answers. Always verify with a small counterexample.
⚠ **Trap**	Wrong sorting key — greedy is very sensitive to sort order (end time vs start time matters)

□ **Trap**	Confusing greedy with DP — greedy never reconsiders; DP does. If you need to reconsider, use DP.

Edge Cases

- No intervals / empty array — return 0
- All intervals overlap — must remove all but one
- Negative jumps in jump game — stuck from the start

Important Practice Problems

- LeetCode 55 — Jump Game
- LeetCode 45 — Jump Game II
- LeetCode 435 — Non-overlapping Intervals
- LeetCode 56 — Merge Intervals
- LeetCode 134 — Gas Station

Real World Python Usage

□ **Real World**	Dijkstra's shortest path algorithm is a greedy algorithm. Huffman coding (used in file compression like gzip) is a greedy algorithm. Operating system job scheduling (Shortest Job First) is greedy. Python's heapq-based priority queue powers greedy algorithms in network routing.

3.15 Union Find (Disjoint Set Union)

The Problem Situation

You need to efficiently group elements, check if two elements belong to the same group, or count the number of connected components. Union Find supports two operations — find (which group does this element belong to?) and union (merge two groups) — both in nearly O(1) with optimisations.

Key Insight

□ **Insight**	Every element starts as its own group. When you union two elements, you merge their groups by making one's root point to the other's root. When you find an element's

	group, you follow parent pointers to the root. Path compression flattens the tree during find. Union by rank keeps the tree shallow.

Analogy

Mental Model	A network of islands. Each island starts disconnected. When you build a bridge between two islands, you merge their groups. Later, when asked 'are island A and island B connected?' you just check if they share the same group root — without tracing every bridge.

Basic Code Template

```
# Union Find — with path compression + union by rank
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank   = [0] * n
        self.count  = n   # number of components

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False   # already connected
        # union by rank
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        self.count -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

✓This Template Solves	LeetCode 547 — Number of Provinces • LeetCode 684 — Redundant Connection • LeetCode 200 — Number of Islands

Example Walkthrough

```
# Example: Number of Provinces — n=3, connections=[[1,2],[1,3]]
# UF init: parent=[0,1,2], count=3
# union(0,1): find(0)=0, find(1)=1 → parent[1]=0, count=2
# union(0,2): find(0)=0, find(2)=2 → parent[2]=0, count=1
# Answer: count = 1 province

# Redundant Connection — find edge that creates a cycle
```

```
# Process edges one by one. If union() returns False,
# the two nodes are already connected → this edge is redundant.
```

Time & Space Complexity

Case	Time	Space
find() with path compression	O(α(n)) ≈ O(1)	O(n)
union() with rank	O(α(n)) ≈ O(1)	
α(n) = inverse Ackermann — practically constant		

Pattern Recognition Clues

- Keywords: connected components, union, same group, redundant edge, number of islands
- You are adding connections incrementally and querying connectivity
- Kruskal's MST algorithm uses Union Find to detect cycles
- Whenever you group elements dynamically — Union Find is the right tool

Interview Traps

□ **Trap**	Forgetting path compression — without it, find() degrades to O(n)
□ **Trap**	Using node values (1-indexed) but initialising parent with range(n) (0-indexed) — causes index error
□ **Trap**	Checking connected before union — union already returns False if already connected

Edge Cases

- Single node — it is its own component
- No edges — every node is its own component
- Self-loop edge — same node unioned with itself

Important Practice Problems

- LeetCode 547 — Number of Provinces
- LeetCode 684 — Redundant Connection
- LeetCode 721 — Accounts Merge

- LeetCode 200 — Number of Islands (alternative approach)
- LeetCode 1319— Number of Operations to Make Network Connected

Real World Python Usage

Real World	Union Find powers Kruskal's MST algorithm used in network design. Social network friend grouping (finding all people in your social cluster) uses Union Find. Image segmentation algorithms use Union Find to group connected pixels. Hadoop's distributed data partitioning uses union-find-like grouping.

3.16 Topological Sort

The Problem Situation

You have a set of tasks with dependencies — task A must complete before task B. You need to find a valid ordering of all tasks, or detect if a circular dependency (cycle) makes ordering impossible. This is only defined for Directed Acyclic Graphs (DAGs).

Key Insight

Insight	A node with no incoming edges (in-degree = 0) has no prerequisites — it can be processed first. Process it, then remove its outgoing edges. Some other nodes will now have in-degree 0. Repeat. If all nodes are processed, a valid topological order exists. If some nodes remain, there is a cycle.

Analogy

Mental Model	A university course schedule. You cannot take Algorithms before taking Data Structures. You cannot take Data Structures before Discrete Math. Topological sort gives you the order to take courses. If two courses are prerequisites of each other, no valid schedule exists — that is a cycle.

Basic Code Template

```
# Kahn's Algorithm — BFS-based topological sort
from collections import defaultdict, deque

def topo_sort(n, prerequisites):
    graph   = defaultdict(list)
    in_deg  = [0] * n
    for a, b in prerequisites:   # b must come before a
```

```
        graph[b].append(a)
        in_deg[a] += 1

    queue = deque([i for i in range(n) if in_deg[i] == 0])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph[node]:
            in_deg[nb] -= 1
            if in_deg[nb] == 0:
                queue.append(nb)

    return order if len(order) == n else []  # [] = cycle detected
```

✅**This Template Solves**	LeetCode 207 — Course Schedule • LeetCode 210 — Course Schedule II • LeetCode 269 — Alien Dictionary

Alternative Template

```
# DFS-based topological sort
def topo_dfs(n, prerequisites):
    graph  = defaultdict(list)
    for a, b in prerequisites:
        graph[b].append(a)
    visited = [0] * n   # 0=unvisited, 1=visiting, 2=done
    order   = []
    def dfs(node):
        if visited[node] == 1: return False  # cycle!
        if visited[node] == 2: return True   # already processed
        visited[node] = 1
        for nb in graph[node]:
            if not dfs(nb): return False
        visited[node] = 2
        order.append(node)
        return True
    for i in range(n):
        if visited[i] == 0:
            if not dfs(i): return []
    return order[::-1]
```

✅**This Template Solves**	LeetCode 210 — Course Schedule II • LeetCode 329 — Longest Increasing Path in Matrix

Example Walkthrough

```
# Example: 4 courses, prerequisites=[[1,0],[2,0],[3,1],[3,2]]
# Graph: 0→1, 0→2, 1→3, 2→3
# in_deg: [0,1,1,2]
```

```
# Queue start: [0] (only node with in_deg=0)
# Process 0: order=[0], reduce in_deg of 1,2 → in_deg=[0,0,0,2]
# Queue: [1,2]
# Process 1: order=[0,1], reduce in_deg of 3 → in_deg=[0,0,0,1]
# Process 2: order=[0,1,2], reduce in_deg of 3 → in_deg=[0,0,0,0]
# Queue: [3]
# Process 3: order=[0,1,2,3]
# len(order)==4==n → valid, return [0,1,2,3]
```

Time & Space Complexity

Case	Time	Space
Kahn's BFS	O(V+E)	O(V+E)
DFS-based	O(V+E)	O(V+E)

Pattern Recognition Clues

- Keywords: prerequisites, dependencies, ordering, schedule, build order, compile order
- Any problem involving 'A must come before B' is topological sort
- Cycle detection in directed graphs — if topological sort fails, there is a cycle
- Alien dictionary — deduce character ordering using topological sort

Interview Traps

□ **Trap**	Building edges in the wrong direction — if [a,b] means 'b before a', then edge goes b→a
□ **Trap**	Returning a partial order when n != len(order) — check this condition to detect cycles
□ **Trap**	Using undirected graph — topological sort is only for directed graphs

Edge Cases

- No prerequisites — all nodes have in-degree 0, any order is valid
- Cycle in graph — impossible to complete, return empty
- Disconnected graph — must process all components

Important Practice Problems

- LeetCode 207 — Course Schedule
- LeetCode 210 — Course Schedule II

- LeetCode 269 — Alien Dictionary
- LeetCode 329 — Longest Increasing Path in a Matrix
- LeetCode 310 — Minimum Height Trees

Real World Python Usage

Real World	Build systems (Make, Bazel, CMake) use topological sort to determine compilation order. Python's pip resolves package dependencies using topological sort. CI/CD pipeline stages use DAG scheduling with topological sort. Apache Airflow (workflow orchestration used in ML pipelines) executes tasks in topologically sorted order.

3.17 Dijkstra's Algorithm

The Problem Situation

You need the shortest path from a source node to all other nodes in a weighted graph where all edge weights are non-negative. BFS gives shortest path in unweighted graphs. Dijkstra extends BFS with a priority queue to handle edge weights.

Key Insight

Insight	Always expand the unvisited node with the smallest known distance first. When you reach a neighbour through this node, check if the new path is shorter than its currently known distance. If yes, update it. A min-heap efficiently finds the next closest node in O(log V).

Analogy

Mental Model	GPS navigation finding the fastest route. You start at your location. The navigation system always explores the closest unvisited road junction next. When it finds a shorter path to a junction via a different route, it updates the best known time. It never revisits a junction once finalised.

Basic Code Template

```
# Dijkstra with heapq — O((V+E) log V)
import heapq
from collections import defaultdict

def dijkstra(graph, start, n):
    dist = [float('inf')] * n
```

```
    dist[start] = 0
    heap = [(0, start)]   # (distance, node)
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue  # outdated entry
        for neighbour, weight in graph[node]:
            new_dist = dist[node] + weight
            if new_dist < dist[neighbour]:
                dist[neighbour] = new dist
                heapq.heappush(heap, (new_dist, neighbour))
    return dist

# Build weighted graph
graph = defaultdict(list)
edges = [(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5)]
for u, v, w in edges:
    graph[u].append((v, w))
    graph[v].append((u, w))  # undirected

print(dijkstra(graph, 0, 4))  # [0,3,1,4]
```

✓This Template Solves	LeetCode 743 — Network Delay Time • LeetCode 1631 — Path with Minimum Effort • LeetCode 778 — Swim in Rising Water

Example Walkthrough

```
# Example: graph=[[0,1,4],[0,2,1],[2,1,2],[1,3,1]]
# dist=[0,inf,inf,inf], heap=[(0,0)]
# Pop (0,0): neighbours (1,4),(2,1)
#   dist[1]=4 → heap=[(1,2),(4,1)]
#   dist[2]=1 → heap=[(1,2),(4,1)]
# Pop (1,2): neighbours (0,1),(1,2),(3,5)
#   0: 1+1=2 > dist[0]=0, skip
#   1: 1+2=3 < dist[1]=4 → dist[1]=3, push (3,1)
#   3: 1+5=6, dist[3]=6, push (6,3)
# Pop (3,1): neighbours (0,4),(2,2),(3,1)
#   dist[3]=3+1=4 → push (4,3)
# Final dist=[0,3,1,4]
```

Time & Space Complexity

Case	Time	Space
Time	O((V+E) log V)	O(V+E)
With Fibonacci heap	O(E + V log V)	

Pattern Recognition Clues

- Keywords: shortest path, weighted graph, minimum cost, network delay, cheapest flight
- Edge weights are non-negative — must use Dijkstra, not BFS
- If edges have negative weights — use Bellman-Ford instead
- Finding path in a grid where movement cost varies — Dijkstra on grid

Interview Traps

□ **Trap**	Not skipping stale heap entries — if d > dist[node]: continue is essential
□ **Trap**	Using Dijkstra with negative weights — gives wrong answer. Use Bellman-Ford instead.
□ **Trap**	Forgetting to initialise dist = [float('inf')] * n before setting dist[start] = 0

Edge Cases

- Disconnected graph — some nodes have dist = infinity at the end
- Single node — dist = [0]
- Target not reachable — return infinity or -1

Important Practice Problems

- LeetCode 743 — Network Delay Time
- LeetCode 1631 — Path With Minimum Effort
- LeetCode 787 — Cheapest Flights Within K Stops
- LeetCode 778 — Swim in Rising Water
- LeetCode 1514 — Path with Maximum Probability

Real World Python Usage

□ **Real World**	Google Maps and Apple Maps use variants of Dijkstra for route planning. Internet routing protocols (OSPF) use Dijkstra to find shortest network paths. Game AI pathfinding uses A* (an informed version of Dijkstra). Python's networkx library provides dijkstra_path() built on this exact algorithm.

3.18 Interval Problems

The Problem Situation

You are given a list of intervals [start, end] and need to merge overlapping intervals, count minimum resources needed, insert a new interval, or find the maximum overlap. Sorting intervals by start time almost always unlocks the greedy or linear solution.

Key Insight

Insight	Sort by start time. Two intervals [a,b] and [c,d] overlap if c <= b (the second starts before the first ends). When they overlap, merge by extending the end: max(b, d). For minimum resources (meeting rooms), use a min-heap tracking end times of active intervals.

Analogy

Mental Model	A calendar with bookings. Sort appointments by start time. If the next appointment starts before the current one ends, they conflict (overlap). To merge overlapping bookings, extend the current booking to the later end time.

Basic Code Template

```
# Merge intervals
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:              # overlap
            merged[-1][1] = max(merged[-1][1], end)
        else:                                   # no overlap
            merged.append([start, end])
    return merged
```

This Template Solves	LeetCode 56 — Merge Intervals • LeetCode 57 — Insert Interval

Alternative Template

```
# Minimum meeting rooms (minimum resources for overlapping intervals)
import heapq
def min_meeting_rooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []   # min-heap of end times
    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heapreplace(heap, end)  # reuse room
        else:
            heapq.heappush(heap, end)     # new room needed
```

```
    return len(heap)

# Insert interval into sorted non-overlapping list
def insert(intervals, new_interval):
    result, i, n = [], 0, len(intervals)
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1
    result.append(new_interval)
    return result + intervals[i:]
```

✅**This Template Solves**	LeetCode 253 — Meeting Rooms II • LeetCode 57 — Insert Interval • LeetCode 435 — Non-overlapping Intervals

Example Walkthrough

```
# Example: merge [[1,3],[2,6],[8,10],[15,18]]
# Sort: already sorted by start
# merged=[[1,3]]
# [2,6]: 2<=3 → overlap, extend: merged=[[1,6]]
# [8,10]: 8>6 → no overlap, append: merged=[[1,6],[8,10]]
# [15,18]: 15>10 → append: merged=[[1,6],[8,10],[15,18]]
```

Time & Space Complexity

Case	Time	Space
Merge intervals	O(n log n)	O(n)
Meeting rooms	O(n log n)	O(n)
Insert interval	O(n)	O(n)

Pattern Recognition Clues

- Keywords: intervals, meetings, overlap, merge, schedule, rooms needed
- Sorting by start time is almost always the first step in interval problems
- Minimum rooms needed = maximum number of overlapping intervals at any point
- Non-overlapping count = total - minimum removals (greedy by end time)

Interview Traps

□ **Trap**	Merging condition: overlap is start <= prev_end, NOT start < prev_end (touching intervals overlap)
□ **Trap**	Forgetting to sort — interval problems almost never work without sorting first
□ **Trap**	Updating end with max but not handling the case where new interval is completely inside old

Edge Cases

- Single interval — return as is
- No overlapping intervals — return original list
- New interval inserted at the beginning or end
- New interval completely inside an existing interval

Important Practice Problems

- LeetCode 56 — Merge Intervals
- LeetCode 57 — Insert Interval
- LeetCode 253 — Meeting Rooms II
- LeetCode 435 — Non-overlapping Intervals
- LeetCode 452 — Minimum Number of Arrows to Burst Balloons

Real World Python Usage

□ **Real World**	Calendar systems (Google Calendar) detect and merge overlapping bookings using interval merging. Database query optimisers merge overlapping index scans. Network bandwidth allocation uses interval scheduling. Python's datetime interval arithmetic uses overlap detection logic.

3.19 Matrix Traversal

The Problem Situation

You are given a 2D grid and need to explore connected regions, find shortest paths, count islands, or traverse in a specific pattern (spiral, diagonal). Matrix problems are essentially graph problems where each cell is a node and adjacent cells are edges.

Key Insight

□ **Insight**	Treat each cell as a node. Movement in 4 directions (up/down/left/right) creates edges. Use DFS for connected region exploration (flood fill, island counting). Use BFS for shortest path in the grid. Always check bounds before visiting a cell.

Analogy

□ **Mental Model**	A city grid of streets. Each intersection is a cell. You can travel North, South, East, or West from any intersection. Blocked cells (walls) are not traversable. Finding the shortest route from one corner to another is BFS on the grid.

Basic Code Template

```
# 4-directional DFS — flood fill / island counting
def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if r<0 or r>=rows or c<0 or c>=cols or grid[r][c]!='1':
            return
        grid[r][c] = '0'   # mark visited (in-place)
        dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                dfs(r, c)
                count += 1
    return count
```

✅**This Template Solves**	LeetCode 200 — Number of Islands • LeetCode 695 — Max Area of Island • LeetCode 733 — Flood Fill

Alternative Template

```
# BFS shortest path on grid
from collections import deque
def shortest_path(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    dirs    = [(0,1),(0,-1),(1,0),(-1,0)]
    visited = set([start])
    queue   = deque([(start[0], start[1], 0)])
    while queue:
        r, c, dist = queue.popleft()
        if (r,c) == end: return dist
        for dr, dc in dirs:
            nr, nc = r+dr, c+dc
            if (0<=nr<rows and 0<=nc<cols and
                grid[nr][nc]!=1 and (nr,nc) not in visited):
```

```
                visited.add((nr,nc))
                queue.append((nr, nc, dist+1))
    return -1

# Spiral order traversal
def spiral_order(matrix):
    result = []
    while matrix:
        result += matrix.pop(0)             # top row
        if matrix and matrix[0]:            # right column
            for row in matrix: result.append(row.pop())
        if matrix: result += matrix.pop()[::-1]  # bottom row
        if matrix and matrix[0]:            # left column
            for row in matrix[::-1]: result.append(row.pop(0))
    return result
```

✅This Template Solves	LeetCode 994 — Rotting Oranges • LeetCode 1091 — Shortest Path in Binary Matrix • LeetCode 54 — Spiral Matrix

Example Walkthrough

```
# Always define directions array — cleaner than 4 if-statements
dirs = [(0,1),(0,-1),(1,0),(-1,0)]   # right, left, down, up
# For 8-directional (including diagonals):
dirs = [(dr,dc) for dr in [-1,0,1] for dc in [-1,0,1] if (dr,dc)!=(0,0)]

# Bounds check pattern
for dr, dc in dirs:
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols:
        # safe to access grid[nr][nc]
        pass
```

Time & Space Complexity

Case	Time	Space
DFS on grid	O(rows × cols)	O(rows × cols)
BFS shortest path	O(rows × cols)	O(rows × cols)
Spiral traversal	O(rows × cols)	O(1)

Pattern Recognition Clues

- Keywords: grid, matrix, island, region, connected, shortest path in grid, flood fill
- Count connected regions — DFS/BFS from each unvisited cell
- Shortest path in grid — always BFS (not DFS)

- Multi-source spread (rotting oranges) — multi-source BFS

Interview Traps

□ **Trap**	Forgetting bounds check — always check 0<=nr<rows and 0<=nc<cols before accessing grid[nr][nc]
□ **Trap**	Modifying the input grid to mark visited — may be forbidden; use a separate visited set instead
□ **Trap**	Using DFS for shortest path in grid — DFS does not guarantee shortest. Use BFS.

Edge Cases

- Empty grid
- Grid with a single cell
- No path exists between start and end
- Grid boundary cells — must not go out of bounds

Important Practice Problems

- LeetCode 200 — Number of Islands
- LeetCode 994 — Rotting Oranges
- LeetCode 1091 — Shortest Path in Binary Matrix
- LeetCode 54 — Spiral Matrix
- LeetCode 695 — Max Area of Island

Real World Python Usage

□ **Real World**	Game development uses grid BFS for pathfinding (A* is an optimised BFS). Robotics uses grid DFS/BFS for maze-solving and navigation. Image processing uses flood fill (grid DFS) for region detection. Geographic information systems (GIS) use grid traversal for terrain analysis. Python's scipy.ndimage uses grid-connected-component algorithms.

Pattern Comparison — Section 3B

Pattern	Best For	Core Idea	Key Tool
Backtracking	All combos/subsets/perms	Choose→explore→unchoose	Recursion + list
Divide & Conquer	Sorting, tree problems	Split, solve, merge	Recursion + @lru_cache
DP (Top-Down)	Overlapping subproblems	Memoize recursive calls	@lru_cache
DP (Bottom-Up)	Optimal substructure	Fill table from base case up	Array / 2D array
Greedy	Interval, scheduling, jumps	Locally optimal = globally optimal	Sort + one pass
Union Find	Connected components, groups	find() + union() in near O(1)	parent[] + rank[]
Topological Sort	Dependency ordering	Process 0 in-degree nodes first	deque + in_deg[]
Dijkstra	Shortest path (weighted)	Min-heap → expand closest first	heapq
Intervals	Overlap, merge, scheduling	Sort by start, merge or count	Sort + heap
Matrix Traversal	Grid regions, shortest grid path	DFS flood fill / BFS shortest	visited set + dirs[]

Master Pattern Recognition Guide

When you read a problem, scan for these keywords and map them to the correct pattern instantly.

If you see this…	Think this pattern
sorted array + pair/triplet	Two Pointers
subarray / substring / contiguous	Sliding Window or Prefix Sum
sum equals k / range sum	Prefix Sum + Hash Map
frequency / count / duplicate / anagram	Hash Map Counting
sorted + find target / boundary	Binary Search
next greater / smaller / histogram	Monotonic Stack
sliding window max/min	Monotonic Queue
linked list cycle / middle	Fast and Slow Pointers

all paths / explore / subtree	DFS
shortest path (unweighted) / level order	BFS
all combinations / subsets / permutations	Backtracking
minimum/maximum / count ways / optimal	Dynamic Programming
interval / merge / schedule / rooms	Greedy + Intervals
connected components / union / group	Union Find
prerequisites / dependencies / ordering	Topological Sort
shortest path (weighted graph)	Dijkstra
grid / island / region / matrix	Matrix DFS or BFS

End of Section 3B. Continue with Section 4 — Pattern Recognition System.